

Automatic Differentiation 与计算图求导

不显式构造 Jacobian，怎样对大型张量程序求导

技术基线：2026-08-22

[返回 Topic Landing Page](#)

Contents

1 本册定位：求导是一种计算问题，不是把公式写得更长	2
2 从数学函数到计算程序：真正需要区分的对象	3
3 朴素方案为什么失败：完整 Jacobian 往往不是我们真正要的对象	4
4 Forward Mode：沿着程序传播输入扰动	5
5 Reverse Mode：从输出向输入反向累积 VJP	5
6 母例：矩阵乘法的 VJP 不需要四阶 Jacobian	6
7 Softmax 的 VJP：从显式 Jacobian 压成两个向量操作	7
8 计算图中的累积：一条边不是一个完整梯度	8
9 Reverse Mode 保存什么：主要内存压力之一来自前向状态	9
10 Custom Operator Contract：框架究竟向 backward 要什么	9
11 正确性、不变量与代价	10
11.1 正确性来自局部线性化和链式法则	10
11.2 Forward mode 与 reverse mode 的成本方向	10
12 最小调用接口：什么时候回到本模型	11
13 一页压缩：从巨大 Jacobian 回到局部线性算子	11
14 资料与版本边界	11

1 本册定位：求导是一种计算问题，不是把公式写得更长

多元微积分已经定义了导数、梯度和 Jacobian。本册不重新拥有这些数学定义，而研究另一个问题：当函数不是一条短公式，而是由矩阵乘法、归一化、激活、索引和控制流组成的大型程序时，怎样以可接受的计算和内存代价得到导数。

Mother Question | 大型程序怎样高效求导

给定由许多局部算子组成的计算程序，怎样利用每个局部算子的线性化与链式法则，只计算下游真正需要的 Jacobian 乘积，而不是显式构造庞大的完整 Jacobian？

本册 Own 的是通用 Automatic Differentiation（自动微分）计算机制：computation graph、forward mode、reverse mode、JVP、VJP 以及保存中间量与重计算之间的代价。神经网络中的 layer-wise credit assignment 属于 Area 60 的 Backprop；拿到梯度后怎样更新参数属于 Area 50 的优化算法。

概念 A	≠	概念 B	真正区别与题目信号	混淆后果
Automatic Differentiation	≠	Symbolic Differentiation	AD 对已执行的基本算子应用链式法则并传播导数信息；符号求导构造新的符号表达式	会把“无需符号化简”误解成“AD 只是在做代数化简”
Automatic Differentiation	≠	Finite Difference	AD 在浮点算术中直接组合局部导数，不引入有限差分的截断误差；有限差分用函数值差商近似导数，并同时受步长选择与舍入误差影响	会把 AD 的数值误差来源和计算成本判断错
Generic reverse-mode AD	≠	Neural-network backpropagation	reverse mode 是通用计算图上的反向导数累积机制；backpropagation 在神经网络语境中调用这一机制，并进一步解释层级信用怎样传回参数	会把通用求导计算与网络学习语义放进同一个 Owner
Gradient Computation	≠	Optimization	AD 负责得到导数信息；optimizer 决定怎样使用这些信息更新参数	会把 ∇L 的计算与 $\theta \leftarrow \theta - \eta \nabla L$ 当成同一步

2 从数学函数到计算程序：真正需要区分的对象

设一个程序实现映射

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad y = f(x).$$

数学上，其局部线性化由 Jacobian $J_f(x) \in \mathbb{R}^{m \times n}$ 描述。但程序并不是以“完整 Jacobian”作为基本对象运行的。它实际拥有：

- **primal value**：前向计算得到的普通值；
- **local operator**：加法、乘法、矩阵乘法、softmax 等局部算子；
- **dependency graph**：哪个中间量由哪些输入产生；
- **local linear map**：当前算子对微小扰动的线性响应；

- **tangent / cotangent**: 沿计算图传播的导数信息。

因此，计算图不是“神经网络专属的数据结构”。只要程序可以被分解为一串可求局部导数的操作，就可以把复合函数写成

$$f = f_k \circ f_{k-1} \circ \cdots \circ f_1.$$

链式法则告诉我们，整体导数由局部线性映射组合而成；AD 的工程问题则是：**按什么方向组合，才能避免生成根本用不到的大矩阵。**

3 朴素方案为什么失败：完整 Jacobian 往往不是我们真正要的对象

如果 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，显式 Jacobian 有 mn 个元素。对一个大张量算子， n 与 m 都可能非常大。

例如

$$Y = XW,$$

其中

$$X \in \mathbb{R}^{N \times D}, \quad Y \in \mathbb{R}^{N \times M}.$$

若把 X 和 Y 展平成向量，那么

$$\frac{\partial \text{vec}(Y)}{\partial \text{vec}(X)} \in \mathbb{R}^{(NM) \times (ND)}.$$

即使这个 Jacobian 具有大量结构，显式物化仍会把一个原本可以用矩阵乘法完成的问题放大成巨型中间对象。

关键转换 | 不要问“Jacobian 是多少”，先问“下游到底要 Jacobian 做什么”

常见的训练与优化任务并不需要逐元素读取完整 Jacobian；真正被下游调用的往往是 Jacobian 对一个向量的作用，或者 Jacobian 转置对一个向量的作用。把“构造矩阵”改成“直接计算线性映射的作用”，是 AD 能扩展到大型程序的关键。

因此出现两个基本计算对象：

$$\text{JVP}(v) := J_f(x)v,$$

以及

$$\text{VJP}(u) := J_f(x)^\top u.$$

JVP 问的是“给输入一个方向扰动 v ，输出沿什么方向变化”；VJP 问的是“给输出一个线性权重/上游梯度 u ，它怎样反向作用到输入”。

4 Forward Mode: 沿着程序传播输入扰动

设

$$z = g(x), \quad y = h(z).$$

给输入方向 $\dot{x}$ ，先计算

$$\dot{z} = J_g(x)\dot{x},$$

再计算

$$\dot{y} = J_h(z)\dot{z}.$$

因此

$$\dot{y} = J_h J_g \dot{x} = J_f \dot{x}.$$



Forward mode 的工作量主要随“需要传播多少个输入方向”增长。因此当输入维度较小、输出维度较大，或者只关心少数输入方向时，它通常更合适。

5 Reverse Mode: 从输出向输入反向累积 VJP

现在考虑标量目标

$$L: \mathbb{R}^n \rightarrow \mathbb{R}.$$

机器学习中常见的是“参数很多，但最终 loss 是一个标量”。如果逐个输入方向做 forward mode，需要重复传播大量 tangent。

Reverse mode 改变方向。仍设

$$z = g(x), \quad L = h(z).$$

定义伴随量 (adjoint)

$$\bar{z} := \frac{\partial L}{\partial z}, \quad \bar{x} := \frac{\partial L}{\partial x}.$$

局部反向规则是

$$\bar{x} = J_g(x)^T \bar{z}.$$

于是计算图从输出向输入执行一串 VJP，而不是显式生成一串 Jacobian。

Reverse-mode 的母模型

Forward pass 计算并保留 backward 需要的 primal state；backward pass 从最终标量的种子梯度 1 开始，对每个局部算子调用 VJP，把上游 cotangent 累积到它的输入。最终得到所有叶子输入相对于 loss 的梯度。

为什么标量 loss 特别适合 reverse mode？因为输出侧只有一个独立方向。一次反向累积就能同时得到很多输入分量的梯度。

6 母例：矩阵乘法的 VJP 不需要四阶 Jacobian

令

$$Y = XW,$$

并设上游梯度

$$\bar{Y} := \frac{\partial L}{\partial Y}.$$

对微扰有

$$dY = dX W + X dW.$$

用 Frobenius 内积表达 loss 的一阶变化：

$$dL = \langle \tilde{Y}, dY \rangle_F.$$

代入得到

$$dL = \langle \tilde{Y}, dX W \rangle_F + \langle \tilde{Y}, X dW \rangle_F.$$

利用迹的循环性质重排：

$$dL = \langle \tilde{Y} W^T, dX \rangle_F + \langle X^T \tilde{Y}, dW \rangle_F.$$

所以

$$\tilde{X} = \tilde{Y} W^T$$

以及

$$\bar{W} = X^T \tilde{Y}.$$

这里没有任何一步需要构造

$$\frac{\partial \text{vec}(Y)}{\partial \text{vec}(X)}.$$

这就是“Jacobian 作为隐式线性算子”的实际含义。

独立检查 | shape 必须和被求导对象一致

若 $X \in \mathbb{R}^{N \times D}$ 、 $W \in \mathbb{R}^{D \times M}$ 、 $\tilde{Y} \in \mathbb{R}^{N \times M}$ ，则 $\tilde{X} \in \mathbb{R}^{N \times D}$ 、 $\bar{W} \in \mathbb{R}^{D \times M}$ 。shape 检查不能证明公式必然正确，但可以快速排除大量转置方向错误。

7 Softmax 的 VJP：从显式 Jacobian 压成两个向量操作

对向量 $p = \text{softmax}(s)$ ，其 Jacobian 为

$$J = \text{diag}(p) - pp^\top.$$

若 backward 已获得上游梯度 $\bar{p}$ ，真正需要的是

$$\bar{s} = J^\top \bar{p}.$$

由于 J 对称，展开得

$$\bar{s} = \text{diag}(p)\bar{p} - p(p^\top \bar{p}).$$

因此

$$\boxed{\bar{s} = p \odot (\bar{p} - \langle p, \bar{p} \rangle 1)}.$$

这条公式展示了一个重要模式：即使完整 Jacobian 是稠密的，只要它有可利用的代数结构，VJP 仍可能只需线性规模的状态和运算。

8 计算图中的累积：一条边不是一个完整梯度

若某个中间变量被多个后续操作使用，例如

$$y = f(x) + g(x),$$

那么 loss 对 x 的梯度是各条依赖路径贡献之和：

$$\bar{x} = \bar{x}_f + \bar{x}_g.$$

因此 reverse mode 的状态更新不是“沿一条边算完就结束”，而是按反向拓扑依赖累计各条下游路径的贡献：

1. 每个下游消费者产生一份对当前节点的 cotangent 贡献；
2. 这些贡献在当前节点做加法累积；
3. 当前节点再用自己的 local VJP 把累计后的 cotangent 传播给输入；
4. 若某个输入被多条路径共享，同样继续累计，直到反向遍历完成。

这里的核心不变量是：某个节点最终保存的 adjoint 等于从所有通向 loss 的计算路径所贡献的一阶线性项之和。

9 Reverse Mode 保存什么：主要内存压力之一来自前向状态

Reverse-mode AD 还需要保存 cotangent、依赖关系或运行时 tape 等状态，但大型张量程序中，一个主要内存来源往往是 backward 所需的 forward 输入、中间值或输出。局部 backward 规则经常需要这些 primal state。例如：

- 乘法 $z = xy$ 的 backward 需要 x, y ；
- softmax 的 VJP 可以直接复用输出 p ；
- 某些 fused operator 可以保存压缩统计量，再在 backward 重算昂贵中间量。

因此 reverse mode 常见权衡是：

save more activations vs recompute more in backward .

保存更多中间量可以减少重复计算，却会提高峰值内存占用，并可能增加写入与读取流量；保存更少则需要在 backward 中重算。Activation checkpointing 和 FlashAttention backward 都利用了这个基本交换，但它们的具体机制由各自 Owner 解释。

Recomputation 什么时候值得

若某个中间量体积很大、重新计算所需 FLOPs 相对便宜，而把它写入并从高层内存重新读取的代价更高，那么 backward 重算可能同时降低峰值显存并缩短运行时间。判断依据不是“重算一定更快”，而是比较目标硬件上 compute、bandwidth、cache 和并行度的相对成本。

10 Custom Operator Contract：框架究竟向 backward 要什么

一个自定义可微算子可以抽象为

$$y = F(x_1, \dots, x_k).$$

其 forward 负责：

1. 计算输出 y ；
2. 只保存 backward 真正需要且值得保存的状态。

其 backward 接收上游 cotangent

$$\bar{y} = \frac{\partial L}{\partial y},$$

并返回

$$\bar{x}_i = \left(\frac{\partial F}{\partial x_i} \right)^\top \bar{y}.$$

也就是说，框架并不要求自定义算子把内部所有中间运算重新暴露给全局 autograd；它只要求算子兑现自己的局部 VJP contract。

这正是 fused operator 可以存在的数学接口：内部可以把多个 primitive operation 合成一个 kernel，只要 forward 数值语义正确、backward 返回与该复合映射一致的 VJP。

11 正确性、不变量与代价

11.1 正确性来自局部线性化和链式法则

AD 不是新的导数定义。其正确性依赖两个条件：

1. 每个 primitive operator 的局部 derivative/JVP/VJP 规则正确；
2. 计算图按照真实依赖关系组合这些局部线性映射。

只要这两点成立，复合得到的导数就是程序所实现映射的导数；若程序包含不可微点，则具体框架还必须定义或选择该点采用的导数约定。

11.2 Forward mode 与 reverse mode 的成本方向

对 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ：

- forward mode 天然一次传播一个输入方向，适合 n 较小或只需少量 JVP；
- reverse mode 从一个输出侧 cotangent seed 开始反向传播；当 m 较小，尤其是 scalar loss 对大量参数求梯度时，通常更合适。

这不是说两者的精确运行时间只由 n, m 决定；真实成本还取决于算子结构、稀疏性、批处理、编译器以及是否能复用中间状态。

12 最小调用接口：什么时候回到本模型

当问题出现以下信号时，应先回到本册：

- “为什么不显式构造 Jacobian?”;
- “JVP 与 VJP 有什么区别?”;
- “为什么标量 loss 通常使用 reverse mode?”;
- “自定义算子的 backward 到底需要返回什么?”;
- “为什么 checkpoint / recomputation 能换显存?”;
- “矩阵乘法或 softmax 的 backward 为什么能写成几个张量操作?”

第一观察层固定为：先写当前映射的输入、输出 shape，再判断下游真正需要 Jv 还是 $J^T v$ 。

若问题进一步变成“梯度怎样穿过神经网络层并影响参数学习”，转交 Area 60 Backprop；若变成“梯度得到以后参数怎样更新”，转交 Area 50 optimization；若变成“Attention fused kernel 怎样重算概率并生成 dQ, dK, dV ”，转交 Attention/Transformer Topic。

13 一页压缩：从巨大 Jacobian 回到局部线性算子

一句话压缩

Automatic Differentiation 把程序看成局部可微算子的组合，并只传播下游需要的 Jacobian 乘积：forward mode 传播 Jv ，reverse mode 传播 $J^T v$ ，从而避免把完整 Jacobian 物化到内存中。

重建本册只需要回答六个问题：

1. 程序的 primal values 和 dependency graph 是什么？
2. 当前真正需要完整 Jacobian，还是只需要它对向量的作用？
3. JVP 与 VJP 分别沿哪个方向传播？
4. 为什么 scalar loss + many parameters 适合 reverse mode？
5. 一个 primitive operator 怎样提供自己的 local VJP？
6. 哪些 forward state 应保存，哪些更适合在 backward 重算？

14 资料与版本边界

本册的数学主干只依赖多元微分与链式法则，不绑定具体深度学习框架版本。框架层面的 custom autograd API、编译器行为和 memory policy 会随版本变化；使用具体 API 时应以当前框架文档为准。